

Study Guide for CSc 4710

Mark Distribution

- 1 page of cheat sheet (handwritten on one side) is allowed
- 3 Questions – 75 Marks
- SQL Query – 30 marks
- Normalization – 30 marks
- Indexing – 15 marks

Compute Attribute Closure

Comp_Attr_closure(X)

{

$closure = X$;

repeat until there is no change: {

if there is an FD $\rightarrow V$ in F such that $U \subseteq closure$

then set $closure = closure \cup V$

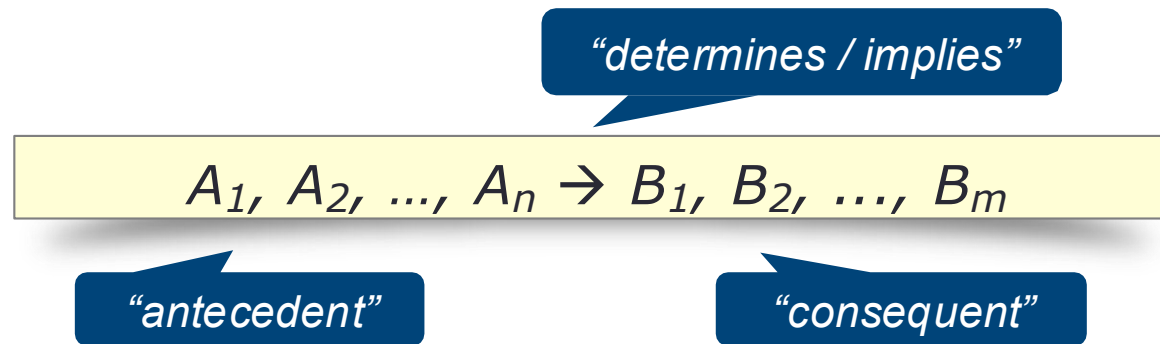
}

return($closure$)

}

Functional Dependencies (FDs)

- Describe data interrelationships
- Constraints on the set of relations
- Determine **how to break/decompose a relation**



- For each pair of tuples t_1 and t_2 in a relation r
 - If two tuples agree on the attributes, $A_1, A_2, \dots, A_n$
 - Then they must also agree on the attributes $B_1, B_2, \dots, B_m$

Functional Dependencies (FDs)

- A relation can have multiple functional dependencies
- Example: schema of relation $R(A, B, C, D)$
FDs: $\{ A \rightarrow B, B \rightarrow C, C \rightarrow CD \}$
- **FD holds** over a relation R if, for **every allowable instance** r of R , r **satisfies the FD**
 - Given some instance r of R , we can check if it **violates** some FD or not
$$t1.X = t2.X \quad \text{but} \quad t1.Y \neq t2.Y$$
 - We cannot tell if FD holds over R by looking at an instance (**cannot prove non-existence of violation**)
 - This is the same for all integrity constraints

Attribute Closure and F^+

- **Attribute Closure (α^+)** = all FDs a particular **attribute** can imply
- **Closure of F (F^+)** = a set of **all** FDs that are implied by F
- An FD f is **logically implied** by a set of FDs F if f holds whenever all FDs in F hold
- To compute the closure of a set of functional dependencies F

```
 $F^+ = F$   
repeat  
  for each functional dependency  $f$  in  $F^+$   
    apply reflexivity and augmentation rules on  $f$   
    add the resulting functional dependencies to  $F^+$   
  for each pair of functional dependencies  $f1$  and  $f2$  in  $F^+$   
    if  $f1$  and  $f2$  can be combined using transitivity  
      then add the resulting functional dependency to  $F^+$   
until  $F^+$  does not change any further
```

Example: Attribute Closure

Let's do this together

Given $R(A,B,C)$

FDs = $\{A \rightarrow B, B \rightarrow C\}$

Compute the attribute closures for all attribute and combination of attributes. Then, think about what can inferred.

	A	B	C	AB	AC	BC	ABC
A	✓	✓	✓	✓	✓	✓	✓
B							
C							
AB							
AC							
BC							
ABC							

Summary: attribute closure
$A^+ = ABC$
$B^+ =$
$C^+ =$
$AB^+ =$
$AC^+ =$
$BC^+ =$
$ABC^+ =$

Example: Attribute Closure

Let's do this together

Given $R(A,B,C)$

FDs = $\{A \rightarrow B, B \rightarrow C\}$

Compute the attribute closures for all attribute and combination of attributes. Then, think about what can inferred.

	A	B	C	AB	AC	BC	ABC
A	✓	✓	✓	✓	✓	✓	✓
B		✓	✓			✓	
C			✓				
AB	✓	✓	✓	✓	✓	✓	✓
AC	✓	✓	✓	✓	✓	✓	✓
BC		✓	✓			✓	
ABC	✓	✓	✓	✓	✓	✓	✓

Summary: attribute closure
$A^+ = ABC$
$B^+ = BC$
$C^+ = C$
$AB^+ = ABC$
$AC^+ = ABC$
$BC^+ = BC$
$ABC^+ = ABC$

Example1: Computing F+

Let's do this together

Given $R(A,B,C,D,E)$

FDs = $\{ A \rightarrow C, B \rightarrow B, C \rightarrow BD, D \rightarrow E \}$

Compute F+

(1) write all LHS
& remaining

  $\rightarrow$
  $\rightarrow$
  $\rightarrow$
  $\rightarrow$
  $\rightarrow$

(2) copy FDs as is

$A \rightarrow C$
 $B \rightarrow B$
 $C \rightarrow B D$
 $D \rightarrow E$
 $E \rightarrow$

(3) apply reflexivity

$A \rightarrow A C$
 $B \rightarrow B$
 $C \rightarrow B C D$
 $D \rightarrow D E$
 $E \rightarrow E$

(4) apply transitivity

$A \rightarrow A B C D E$
 $B \rightarrow B$
 $C \rightarrow B C D E$
 $D \rightarrow D E$
 $E \rightarrow E$

$F^+ = \{ A \rightarrow ABCDE, B \rightarrow B, C \rightarrow BCDE, D \rightarrow DE, E \rightarrow E \}$

Example2: Computing F+

Let's do this together

Given $R(A,B,C,D,E)$

FDs = $\{A \rightarrow BC, B \rightarrow D, CD \rightarrow E\}$

Compute F+

Decompose: $A \rightarrow A, A \rightarrow B, A \rightarrow C$

(1) write all LHS
& remaining

A $\rightarrow$
B $\rightarrow$
C $\rightarrow$
D $\rightarrow$
E $\rightarrow$
CD $\rightarrow$

(2) copy FDs as is

 $\rightarrow$ BC
 $\rightarrow$ D
 $\rightarrow$
 $\rightarrow$
 $\rightarrow$
CD $\rightarrow$ E

(3) apply reflexivity

 $\rightarrow$ ABC
 $\rightarrow$ B D
 $\rightarrow$ C
 $\rightarrow$ D
 $\rightarrow$ E
CD $\rightarrow$ CDE

(4) apply transitivity

 $\rightarrow$ ABCD
 $\rightarrow$ B D
 $\rightarrow$ C
 $\rightarrow$ D
 $\rightarrow$ E
CD $\rightarrow$ CDE

Example2: Computing F+ (cont.)

Let's do this together

Decompose: $A \rightarrow A, A \rightarrow B, A \rightarrow C, A \rightarrow D$
Union: $A \rightarrow C$ and $A \rightarrow D$, then $A \rightarrow CD$

(from previous page)

(4) apply transitivity

 $\rightarrow ABCD$

 $\rightarrow B D$

 $\rightarrow C$

 $\rightarrow D$

 $\rightarrow E$

$CD \rightarrow CDE$

(5) apply transitivity

 $\rightarrow ABCDE$

 $\rightarrow B D$

 $\rightarrow C$

 $\rightarrow D$

 $\rightarrow E$

$CD \rightarrow CDE$

$F^+ = \{ A \rightarrow ABCDE, B \rightarrow BD, C \rightarrow C, D \rightarrow D, E \rightarrow E, CD \rightarrow CDE \}$

Calculate candidate key of relation R

Definition:

In the relation R with attribute set U , where X is a subset of U , if $X \rightarrow U$ holds in R , then X is referred to as a **superkey** of R .

If $X \rightarrow U$ holds in R and for any proper subset X_i of X , $X_i \rightarrow U$ does not hold, then X is called a **candidate key** for R .

Solving process: Calculate candidate key of relation R

1. Given the relation $R(U, F)$, categorize all attributes of R into four classes: L, R, LR, and N. In this categorization:

L represents attributes that appear only on the left side of functional dependencies.

R represents attributes that appear only on the right side of functional dependencies.

LR represents attributes that appear on both the left and right sides of functional dependencies.

N represents attributes that do not appear on either side of functional dependencies.

2. Let $X = L \cup N$, and $Y = LR$. Calculate $(X)^+$. If $(X)^+$ contains all attributes of R , then X is a unique candidate key for R . Proceed to step 5.

3. From the attributes in Y , select any attribute A and compute $(XA)^+$. If it includes all attributes of R , it is a candidate key. Repeat this process with other attributes from Y until all attributes are tested.

4. If you have found all candidate keys, proceed to step 5. If not, select 2, 3, and more attributes from Y one by one and compute their closures. If the closure includes all attributes of R , it is a candidate key.

5. Finish the algorithm and output the candidate keys.

Example 1: Calculate candidate key of relation R

Consider the relation $R(A, B, C, D)$ and the FD set $F=\{B\rightarrow C, D\rightarrow A\}$.

List all Candidate Keys for R .

$$L = \{B, D\}$$

$$R = \{C, A\}$$

$$LR = \emptyset$$

$$N = \emptyset$$

$$X = L \cup N = \{B, D\}$$

$$(BD)^+ = \{BDCA\}$$

Because $(BD)^+$ contains all attributes of R .

Thus $\{B, D\}$ a unique candidate key for R .

Normalization

- **Normalization** = technique of organizing data in a database
- Two purposes:
 - **Eliminating redundant data**
 - Avoid storing the same data in multiple tables
 - **Ensuring data dependencies** make sense
 - Store data logically – only related data in a table, nothing else
- Need to **refine schema**

Third Normal Form (3NF)

- 2NF + lossless-join + **dependency preserving**
- For every non-trivial dependency, **either LHS is a superkey** or the **RHS consists of prime attributes only**

No transitive dependencies

computingID	course	grade	textbook_id	title
ht1y	cs1	B+	book1	Intro to Python
dt2y	cs1	A-	book1	Intro to Python
dt2y	cs2	A	book2	Intro to Java
md3y	cs1	A	book1	Intro to Python
mn4e	cs2	B	book2	Intro to Java
md3y	cs2	A	book2	Intro to Java

B → D ?

Suppose we want to keep track of textbook_id and title for the course

Let's simplify the name: R (A, B, C, D, E), AB is a candidate key

FDs { $AB \rightarrow C$, $AB \rightarrow D$, $D \rightarrow E$ }

Since $AB \rightarrow D$, $D \rightarrow E$, D and E are non keys -- "**transitive dependency**"

Problem with dependency ... Fix it!

Third Normal Form (3NF)

To convert the table into 3NF, decompose the table to remove transitive dependency

computingID	course	grade	textbook_id	title
ht1y	cs1	B+	book1	Intro to Python
dt2y	cs1	A-	book1	Intro to Python
dt2y	cs2	A	book2	Intro to Java

computingID	course	grade
ht1y	cs1	B+
dt2y	cs1	A-
dt2y	cs2	A
md3y	cs1	A
mn4e	cs2	B
md3y	cs2	A

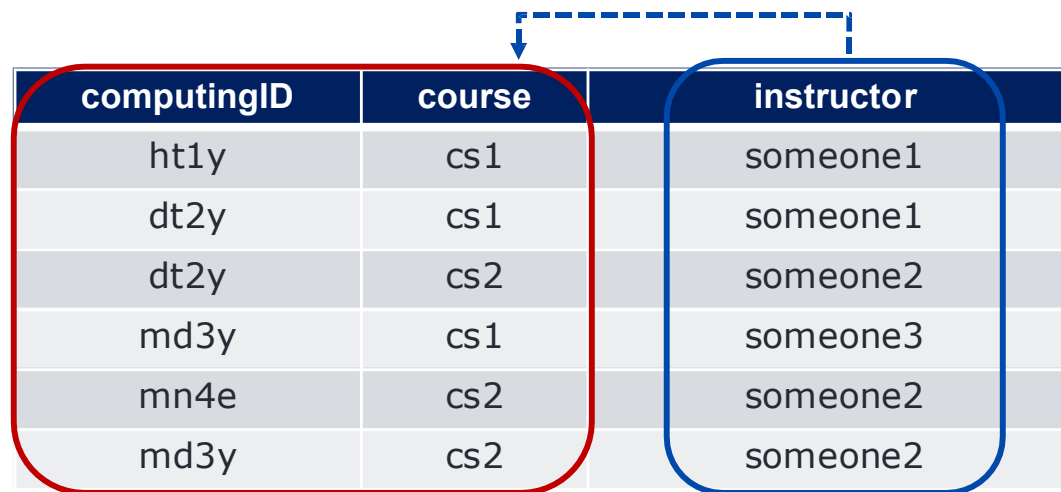
course	textbook_id	title
cs1	book1	Intro to Python
cs2	book2	Intro to Java

Ensure data integrity; no transitive dependency;
dependency is in the same relation

Preserve all FDs but allow anomalies (may have redundancy)

Boyce-Codd Normal Form (BCNF)

- 1NF + lossless-join + redundant free
- For every non-trivial dependency, $X \rightarrow A$, X is a **superkey**.



computingID	course	instructor
ht1y	cs1	someone1
dt2y	cs1	someone1
dt2y	cs2	someone2
md3y	cs1	someone3
mn4e	cs2	someone2
md3y	cs2	someone2

All dependencies must be from full key

Suppose we know that $\text{instructor} \rightarrow \text{course}$

Cannot have a non-key implies a key

Let's simplify the name: $R(A, B, C)$, AB is a candidate key
FDs $\{ AB \rightarrow C, C \rightarrow B \}$

Since $C \rightarrow B$, non-key implies a (part of) key

Not satisfy BCNF -- Fix it!

Boyce-Codd Normal Form (BCNF)

To convert the table into BCNF, decompose the table to remove non-keys that imply a key – to make all dependencies from a key

computingID	course	instructor
ht1y	cs1	someone1
dt2y	cs1	someone1
dt2y	cs2	someone2
		someone3
ht1y	someone1	someone2
dt2y	someone1	someone2
dt2y	someone2	
md3y	someone3	
mn4e	someone2	

Given
FDs { AB $\rightarrow$ C, C $\rightarrow$ B }

lost FD

instructor	course
someone1	cs1
someone2	cs2

Remove redundant data; ensure data integrity; may have dependency across relations (need to join to check dependency)

No transitive FDs, no non-key dependencies, but can lose FDs

Example

Let's consider the `work_project` table that stores information about projects, assignment, and budget and hour allocation

`work_project`

prjName	prjMgr	empId	hours	eName	budget	startDate	salary	empMgr	empDept	rating
Jupiter	Smith	E101	25	Jones	100000	01/15/19	60000	Levine	10	9
Jupiter	Smith	E105	40	Adams	100000	01/15/19	55000	Jones	12	
Jupiter	Smith	E110	10	Rivera	100000	01/15/19	48000	Levine	10	8
Maxima	Lee	E101	15	Jones	200000	03/15/18	60000	Levine	10	
Maxima	Lee	E110	30	Rivera	200000	03/15/18	48000	Levine	10	
Maxima	Lee	E120	15	Tanaka	200000	03/15/18	45000	Jones	15	

Notice that the table consists of many attributes, some data are redundant.

Is work_project Table in 3NF?

work_project

prjName	prjMgr	empId	hours	eName	budget	startDate	salary	empMgr	empDept	rating
Jupiter	Smith	E101	25	Jones	100000	01/15/19	60000	Levine	10	9
Jupiter	Smith	E105	40	Adams	100000	01/15/19	55000	Jones	12	
Jupiter	Smith	E110	10	Rivera	100000	01/15/19	48000	Levine	10	8
Maxima	Lee	E101	15	Jones	200000	03/15/18	60000	Levine	10	
Maxima	Lee	E110	30	Rivera	200000	03/15/18	48000	Levine	10	
Maxima	Lee	E120	15	Tanaka	200000	03/15/18	45000	Jones	15	

2NF  [Project(prjName, prjMgr, budget, startDate)
Emp(empId, empName, salary, empMgr, empDept)
Work(prjName, empID, hours, rating)

Use the set of tables Project, Emp, and Work. Check if there is a non-key attribute or combination of attribute that functionally determines another non-key attribute.

Check if there is a transitive dependency.

Is work_project Table in 3NF?

2NF  $\left[\begin{array}{l} \text{Project}(\underline{\text{prjName}}, \text{prjMgr}, \text{budget}, \text{startDate}) \\ \text{Emp}(\underline{\text{empId}}, \text{empName}, \text{salary}, \text{empMgr}, \text{empDept}) \\ \text{Work}(\underline{\text{prjName}}, \underline{\text{empId}}, \text{hours}, \text{rating}) \end{array} \right.$

$\text{prjName} \rightarrow \text{prjMgr}, \text{budget}, \text{startDate}$
 $\text{empId} \rightarrow \text{empName}, \text{salary}, \text{empMgr},$
 $\text{empDept} \text{ prjName}, \text{empId} \rightarrow \text{hours},$
 rating
 $\text{empDept} \rightarrow \text{empMgr}$

Given FDs

Check if there is a transitive dependency.

Project: No transitive dependency in Project. Thus, it is in 3NF.

Emp: There is a transitive dependency since $\text{empDept} \rightarrow \text{empMgr}$.

Since empDept is not a superkey, nor is empMgr part of a candidate key, this violates 3NF. Thus, we need to decompose Emp

$\text{Emp}(\underline{\text{empId}}, \text{empName}, \text{salary}, \text{empDept})$
 $\text{Department}(\underline{\text{empDept}}, \text{empMgr})$

Work: No transitive dependency in Work. Thus, it is in 3NF.

Use 3NF, Decompose work_project

2NF  [Project(prjName, prjMgr, budget, startDate)
Emp(empId, empName, salary, empMgr, empDept)
Work(prjName, empID, hours, rating)

prjName $\rightarrow$ prjMgr, budget, startDate
empId $\rightarrow$ empName, salary, empMgr,
empDept prjName, empId $\rightarrow$ hours,
rating
empDept $\rightarrow$ empMgr

Given FDs

Therefore, the new set of 3NF tables is

Project(prjName, prjMgr, budget, startDate)
Emp(empId, empName, salary, empDept)
Department(empDept, empMgr)
Work(prjName, empID, hours, rating)

Is work_project Table in BCNF?

work_project

prjName	prjMgr	empId	hours	eName	budget	startDate	salary	empMgr	empDept	rating
Jupiter	Smith	E101	25	Jones	100000	01/15/19	60000	Levine	10	9
Jupiter	Smith	E105	40	Adams	100000	01/15/19	55000	Jones	12	
Jupiter	Smith	E110	10	Rivera	100000	01/15/19	48000	Levine	10	8
Maxima	Lee	E101	15	Jones	200000	03/15/18	60000	Levine	10	
Maxima	prjName → prjMgr, budget, startDate Given FDs empId → empName, salary, empMgr, empDept prjName, empId → hours, rating empDept → empMgr									
Maxima										

Check if there is a determinant that is not a superkey.

Any one of empId, empDept, or prjName is sufficient to show that the work_project table is not BCNF. Thus, normalize it.

Use BCNF, Decompose work_project

work_project

prjName	prjMgr	empId	hours	eName	budget	startDate	salary	empMgr	empDept	rating
Jupiter	Smith	E101	25	Jones	100000	01/15/19	60000	Levine	10	9
Jupiter	Smith	E105	40	Adams	100000	01/15/19	55000	Jones	12	
Jupiter	Smith	E110	10	Rivera	100000	01/15/19	48000	Levine	10	8
Maxima	Lee	E101	15	Jones	200000	03/15/18	60000	Levine	10	
Maxima										
Maxima										

prjName → prjMgr, budget, startDate

Given FDs

empId → empName, salary, empMgr,
empDept prjName, empId → hours,
rating

empDept → empMgr

Therefore, the new set of BCNF tables is

Project(prjName, prjMgr, budget, startDate)

Emp(empId, empName, salary, empDept)

Department(empDept, empMgr)

Work(prjName, empID, hours, rating)

Consider each table. For every nontrivial FD, a determinant determines everything and thus it is a superkey.

Calculate minimal set of functional dependencies

A minimal cover of a set of functional dependencies (FD) E is a minimal set of dependencies F that is equivalent to E .

A set of FD F to be minimal if it satisfies the following conditions –

Every dependency in F has a single attribute for its right-hand side.

We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X , and still have a set of dependencies that is equivalent to F .

We cannot remove any dependency from F and still have a set of dependencies that are equivalent to F .

Dependency preserving

Dependency Preserving Decomposition is a technique used in DBMS to decompose a relation into smaller relations while preserving the functional dependencies between the attributes. The goal is to improve the efficiency of the database by reducing redundancy and improving query performance.

The 1NF, 2NF, and 3NF are valid for dependency-preserving decomposition.

If we decompose a relation R into relations R_1 and R_2 , All dependencies of R either must be a part of R_1 or R_2 or must be derivable from a combination of functional dependency of R_1 and R_2 .

For Example, A relation $R(A, B, C, D)$ with FD set $\{A \rightarrow BC\}$ is decomposed into $R_1(ABC)$ and $R_2(AD)$ which is dependency preserving because FD $A \rightarrow BC$ is a part of $R_1(ABC)$.

Lossless join

To check for lossless join decomposition using the FD set, the following conditions must hold:

1. The Union of Attributes of R1 and R2 must be equal to the attribute of R. Each attribute of R must be either in R1 or in R2.

$$\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$$

2. The intersection of Attributes of R1 and R2 must not be NULL.

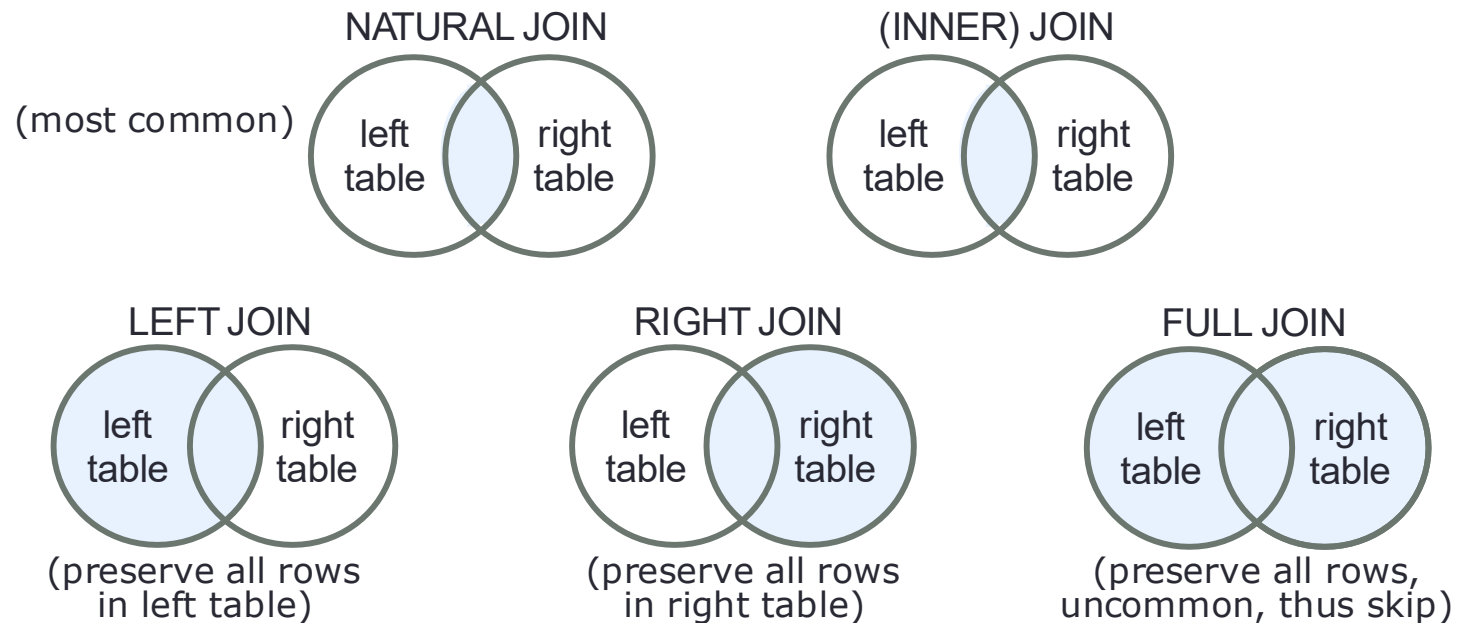
$$\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$$

3. The common attribute must be a key for at least one relation (R1 or R2)

$$\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R1) \text{ or } \text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R2)$$

JOIN

- **Combine** related tables or sets of data on key values
- All tables (or relations) must have a unique identifier (**primary key**)
- Any related tables must contain a copy of the unique identifier (**foreign key**) from the related table



SQL – Join

- A Join is a Query that combines data from multiple tables
 - Multiple tables are specified in the From Clause
 - For two tables to be joined in a sensible manner, they need to have data in common

Example:

Schema: Movies (movie_title, director_id, release_date)
 People(person_fname, person_lname, person_id)

Query: Select movie_title, person_fname, person_lname
 From Movies, People
 Where director_id = person_id

SQL – Joining Condition

- For a useful Join query a joining condition is required
 - Defined in where clause as relationships between columns
 - Multiple conditions may be defined if multiple columns shared
 - More than two tables can be joined in a query

Example: Find people who live in same state as studio

Schema:

Studios(studio_id, studio_state, studio_name, studio_city)

People(person_fname, person_lname, person_id, person_state, person_city)

Query:

Select person_fname, person_lname, studio_name

From Movies, People

Where studio_city = person_city

AND studio_state = person_state

SQL – More than two tables

Example: Get title, director, studio, city for all movies in the database

Schema:

Studios(studio_id, studio_state, studio_name, studio_city)

People(person_fname, person_lname, person_id, person_state, person_city)

Movies(movie_title, director_id, studio_id)

Query:

Select M.movie_title, M.studio_id, P.person_fname, P.person_lname, S.studio_city

From Movies M, People P, Studio S

Where M.director_id = P.person_id

AND M.studio_id = S.studio_id

SQL – Self Join

- Required to compare values within a single column
 - Need to define aliases for the table names

Example: Find actors living in the same state

Schema:

People(person_fname, person_lname, person_id, person_state, person_city)

Query:

```
Select p1.person_id, p1.person_fname, p1.person_lname, p1.person_state  
From People p1, People p2  
Where p1.person_state = p2.person_state  
AND p1.person_id != p2.person_id
```

Note: Distinct operator is critical because if there are more than two people from any state each person will appear as many times as there are people from the state

SQL – Join

- More verbose than previous versions of SQL
 - Need to define aliases for the table names
- Separates the condition for joining from condition for filtering

Example: Find actors living in the same state

Schema:

People(person_fname, person_lname, person_id, person_state, person_city)
Movies(movie_title, director_id, studio_id)

Query:

```
Select movie_title, person_fname, person_lname  
From Movies INNER JOIN People  
ON director_id = person_id
```

```
Select movie_title, person_fname, person_lname  
From Movies INNER JOIN People  
ON director_id = person_id  
Where studio_id = 1
```

SQL – Multiple Table Join

Example: Get title, director, studio, city for all movies in database

Schema:

Studios(studio_id, studio_state, studio_name, studio_city)

People(person_fname, person_lname, person_id, person_state, person_city)

Movies(movie_title, director_id, studio_id)

Query:

Select Movies.movie_title, Movies.studio_id, Person.person_fname, Person.person_lname,
Studio.studio_city

From (People Inner Join

(Movies Inner Join Studio

On Studio.studio_id = Movie.studio_id)

On Movie.director_id = Person.person_id

SQL – Left/Right Join

Example:

Schema:

People(person_fname, person_lname, person_id, person_state, person_city)

Movies(movie_id, movie_title, director_id, studio_id)

Location(movie_id, city, state)

Query:

Select movie_title, city, state

From Movies Left Join Locations

On Movies.movie_id = Locations.movie_id

Includes all
non matched
movie titles

Select movie_title, person_fname, person_lname

From Movies Right Join People

On Movies.director_id = Person.person_id

Includes
all people
not matching
to directors

Let's Try: Join

For each sailor, find the number of boats they have reserved
(assume we will display sname and the number of boats)

Sailors

sid	sname	rating	age
22	Yuppy	9	35
31	Lubber	8	55.5
44	Guppy	5	35
48	Ole Red	8	92.3
58	Rusty	10	40

Reserves

sid	bid	day
22	101	2003-06-05
22	104	2003-06-15
44	102	2003-06-05
48	105	2003-06-14
58	103	2003-06-07

```
SELECT s.sname, COUNT(r.bid) AS num_of_boats_reserved
FROM Sailors s
LEFT JOIN Reserves r ON s.sid = r.sid
GROUP BY s.sname;
```

(Challenging) Example: Self Joins

Find names of all TAs who are assigned to Database Systems and Software Analysis

Hiring(TA_id, Name, Year, Two_week_hours)

TA_id	Name	Year	Two_week_hours
1234	Minnie	4	20
2345	Humpty	3	24
3456	Dumpty	4	30
3333	Minnie	3	12
5678	Mickey	2	16

TA_Assignment(TA_id, Course)

TA_id	Course
1234	Database Systems
2345	Database Systems
3456	Cloud Computing
1234	Software Analysis
5678	Web Programming Lang.
2345	Software Analysis

How should we write SQL?

(Challenging) Example: Self Joins

Find names of all TAs who are assigned to Database Systems and Software Analysis

Hiring(TA_id, Name, Year, Two_week_hours)

TA_id	Name	Year	Two_week_hours
1234	Minnie	4	20
2345	Humpty	3	24
3456	Dumpty	4	30
3333	Minnie	3	12
5678	Mickey	2	16

TA_Assignment(TA_id, Course)

TA_id	Course
1234	Database Systems
2345	Database Systems
3456	Cloud Computing
1234	Software Analysis
5678	Web Programming Lang.
2345	Software Analysis

```
SELECT H.Name
From   Hiring AS H, TA_Assignment AS T
WHERE  H.TA_id = T.TA_id AND
        T.Course = "Database Systems" AND
        T.Course = "Software Analysis";
```

Will this work?

No.
Returns empty set

(Challenging) Example: Self Joins

Find names of all TAs who are assigned to Database Systems and Software Analysis

Hiring(TA_id, Name, Year, Two_week_hours)

TA_id	Name	Year	Two_week_hours
1234	Minnie	4	20
2345	Humpty	3	24
3456	Dumpty	4	30
3333	Minnie	3	12
5678	Mickey	2	16

TA_Assignment(TA_id, Course)

TA_id	Course
1234	Database Systems
2345	Database Systems
3456	Cloud Computing
1234	Software Analysis
5678	Web Programming Lang.
2345	Software Analysis

```
SELECT H.Name
From   Hiring AS H, TA_Assignment AS T1,
        TA_Assignment AS T2
WHERE  H.TA_id = T1.TA_id AND
        H.TA_id = T2.TA_id AND
        T1.Course = "Database Systems" AND
        T2.Course = "Software Analysis";
```

Will this work?

Name
Minnie
Humpty

(Challenging) Example: Self Joins

Find names of all TAs who are assigned to Database Systems and Software Analysis

Hiring(TA_id, Name, Year, Two_week_hours)

TA_id	Name	Year	Two_week_hours
1234	Minnie	4	20
2345	Humpty	3	24
3456	Dumpty	4	30
3333	Minnie	3	12
5678	Mickey	2	16

TA_Assignment(TA_id, Course)

TA_id	Course
1234	Database Systems
2345	Database Systems
3456	Cloud Computing
1234	Software Analysis
5678	Web Programming Lang.
2345	Software Analysis

```
SELECT H.Name
From   Hiring AS H, TA_Assignment AS T1,
        TA_Assignment AS T2
WHERE  H.TA_id = T1.TA_id AND
        H.TA_id = T2.TA_id AND
        T1.Course = "Database Systems" AND
        T2.Course = "Software Analysis";
```

Let's consider each part of the query

All pairs of courses a TA is assigned

For each pair, check for courses

Example: JOIN

Find the total number of unique sailors who have reserved each boat (ordered the number of sailors in descending order).

Display the count, boat name, and boat id

```
Boats (bid, bname, color)
Sailors (sid, sname, rating, age)
Reserves (sid, bid, day)
```

```
SELECT b.bid, b.bname, COUNT(DISTINCT r.sid) AS sailor_count
FROM Boats b
LEFT JOIN Reserves r ON b.bid = r.bid
GROUP BY b.bid, b.bname;
```

Example: JOIN

Find the average age of sailors who have reserved each boat.
Show boat name, boat id, and the average age.

Order results by boat id.

```
Boats (bid, bname, color)
```

```
Sailors (sid, sname, rating, age)
```

```
Reserves (sid, bid, day)
```

```
SELECT b.bid, b.bname, AVG(s.age) AS average_age
FROM Boats b
LEFT JOIN Reserves r ON b.bid = r.bid
LEFT JOIN Sailors s ON r.sid = s.sid
GROUP BY b.bid, b.bname
ORDER BY b.bid;
```

Example: JOIN

Find the average age of sailors who have reserved each boat? Show boat name, bid, and the average age. Order results by bid. (*from Recap 2*)

In addition, only show the boat info where the average age of sailors who have reserved that boat is > 35 years old.

```
Boats (bid, bname, color)
```

```
Sailors (sid, sname, rating, age)
```

```
Reserves (sid, bid, day)
```

```
SELECT b.bid, b.bname, AVG(s.age) AS average_age
FROM Boats b
LEFT JOIN Reserves r ON b.bid = r.bid
LEFT JOIN Sailors s ON r.sid = s.sid
GROUP BY b.bid, b.bname
HAVING AVG(s.age) > 35
ORDER BY b.bid;
```

Self Join

Find all students (sid) who live in the same city and on the same street as their mentor

```
Mentorship (mentee_sid, mentor_sid)
    -- mentor_sid is a mentor of another student mentee_sid
Study (sid, credits)  -- credits the student has taken
Enrollment (dept_id, sid) -- dept the student is enrolled in
Student (sid, street, city) -- street, city the student lives
```

```
SELECT DISTINCT m.mentor_sid AS mentor, s1.sid AS student
FROM Mentorship m
JOIN Student s1 ON m.mentee_sid = s1.sid
JOIN Student s2 ON m.mentor_sid = s2.sid
WHERE s1.street = s2.street
    AND s1.city = s2.city;
```

SQL-subqueries

Subquery is a query that is embedded in WHERE clause of another SQL query. Important rules for Subqueries:

- You can place the Subquery in a number of SQL clauses: WHERE clause, HAVING clause, FROM clause. Subqueries can be used with SELECT, UPDATE, INSERT, DELETE statements along with expression operator. It could be equality operator or comparison operator such as =, >, <, <= and Like operator.
- A subquery is a query within another query. The outer query is called as main query and inner query is called as subquery.
- The subquery generally executes first when the subquery doesn't have any co-relation with the main query, when there is a co-relation the parser takes the decision on the fly on which query to execute on precedence and uses the output of the subquery accordingly.
- Subquery must be enclosed in parentheses.
- Subqueries are on the right side of the comparison operator.
- ORDER BY command cannot be used in a Subquery. GROUPBY command can be used to perform same function as ORDER BY command.
- Use single-row operators with single row Subqueries. Use multiple-row operators with multiple-row Subqueries.

Subqueries in WHERE

Return a **single value** or a **relation** that can be compared to another value in a WHERE clause

Find the name(s) of the employee(s) who earn the highest salary for each job

```
HW_emp(empno, ename, job, sal)
```

```
SELECT E1.ename  
FROM HW_emp E1  
WHERE E1.sal = (SELECT MAX(E2.sal)  
                FROM HW_emp E2  
                WHERE E1.job = E2.job);
```

"Correlated" query

Subqueries in WHERE

Return a **single value** or a **relation** that can be compared to another value in a WHERE clause

Find the name(s) of the instructor(s) who earn the highest salary

```
Instructor (dept_name, ID, name, salary)
```

```
SELECT name
FROM   instructor
WHERE  salary = (SELECT MAX(salary)
                  FROM instructor);
```

"Uncorrelated" query

Subqueries in WHERE

Can be used to evaluate existential or universal quantifiers

At least one element

All elements

Any relational operators

Values of a column

ANY ($\exists$)

ALL ($\forall$)

IN

NOT IN

EXISTS

NOT EXISTS

SELECT ... WHERE **constant** > ANY (subquery);

SELECT ... WHERE **constant** > ALL (subquery);

SELECT ... WHERE **attribute** IN (subquery);

SELECT ... WHERE **attribute** NOT IN (subquery);

SELECT ... WHERE EXISTS (subquery);

SELECT ... WHERE NOT EXISTS (subquery);

Existential Quantifier (EXISTS)

```
account (account number, branch_name, balance)
borrower (customer_name, loan_number)
branch (branch name, branch_city, assets)
depositor (customer_name, account_number)
loan (loan number, branch_name, amount)
```

Find the names of the branches that have **some** customers who have both loan(s) and account(s) from the bank

```
SELECT T1.branch_name
FROM account T1
WHERE EXISTS
    (SELECT *
     FROM depositor D NATURAL JOIN borrower B
     WHERE D.account_number = T1.account_number),
```

tuple(s)

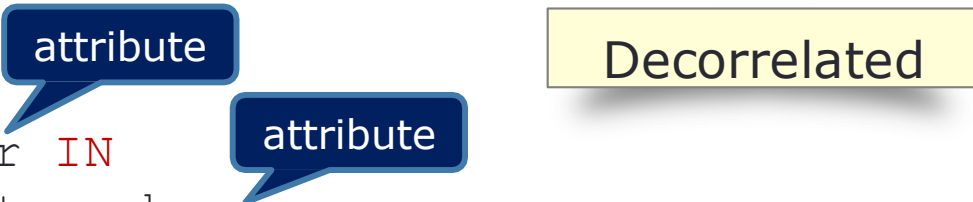
Correlated

Existential Quantifier (IN)

```
account (account number, branch_name, balance)
borrower (customer_name, loan number)
branch (branch name, branch_city, assets)
depositor (customer_name, account number)
loan (loan number, branch_name, amount)
```

Find the names of the branches that have **some** customers who have both loan(s) and account(s) from the bank

```
SELECT T1.branch_name
FROM account T1
WHERE T1.account_number IN
      (SELECT D.account_number
       FROM depositor D NATURAL JOIN borrower B);
```



IN and Set Membership

takes (ID, course_id, sec_id, semester, year, grade)

I need to know who takes the following courses:
CS-101, CS-315, BIO-101

```
SELECT *  
FROM takes  
WHERE course_id IN  
      ('CS-101', 'CS-315', 'BIO-101');
```

attribute

Set of literal values



```
SELECT *  
FROM takes  
WHERE course_id='CS-101' OR course_id='CS-315' OR  
      course_id='BIO-101';
```

Universal Quantifier (NOT IN)

Product(pid, name, cid)

Company(cid, cname, city)

Customer(custId, name, city)

Purchase(purchase date, pid, custId, quantity, price)

Solution #1

Find the names of the customers who made purchases that are
> \$1000 **only**

Step 1: Find the customers who make **some** purchases <= 1000

```
SELECT DISTINCT T1.name
FROM Customer T1
WHERE T1.custId IN (SELECT T2.custId
                    FROM Purchase T2
                    WHERE T2.price <= 1000)
```

Universal Quantifier (NOT IN)

Find the names of the customers who made purchases that are
> \$1000 **only**

Step 1: Find the customers who make **some** purchases <= 1000

```
SELECT DISTINCT T1.name
FROM Customer T1
WHERE T1.custId IN (SELECT T2.custId
                    FROM Purchase T2
                    WHERE T2.price <= 1000)
```

Step 2: Find **all** customers who make purchase > 1000

```
SELECT DISTINCT T1.name
FROM Customer T1 NATURAL JOIN Purchase P
WHERE T1.custId NOT IN (SELECT T2.custId
                       FROM Purchase T2
                       WHERE T2.price <= 1000)
```

Universal Quantifier (NOT EXISTS)

Product(pid, name, cid)

Company(cid, cname, city)

Customer(custId, name, city)

Purchase(purchase date, pid, custId, quantity, price)

Solution #2

Find the names of the customers who made purchases that are
> \$1000 **only**

```
SELECT DISTINCT T1.name
FROM Customer T1 NATURAL JOIN purchase
WHERE NOT EXISTS (SELECT *
                   FROM purchase T2
                   WHERE T1.custId = T2.custId AND
                        T2.price <= 1000)
```

What is Indexing?

- A **“labeled” pointer** to an (a collection of) **item** that satisfies some common property
- Examples in the Real World?

Table of Contents

Foreword	xiii
I What Is MRO Procurement?	1
II Prerequisites for Success	11
III Database Construction and Maintenance	25
IV Partnering Your Way to Success	41
V Saving Hard Dollars	51
VI Saving Soft Dollars	83
VII Selling the Program	115
VIII Documenting Savings	127
IX A Top Management View	151
APPENDIX I Typical MRO Product Categories	161
APPENDIX II Worksheets for Documenting MRO Cost Savings	167
APPENDIX III Letter from Major Automotive Builder to Suppliers	173
APPENDIX IV Letter from Supplier to Customer Describing Range of Value-added Services and Findings of MRO Survey	181

Index

A
abuse 9
Abyssinian 12
accidents 61
acetaminophen 122
acquiring a cat 9, 19
adjusting to changes 68
adopting a cat 1–28
adult cats
 adopting 15
aggression 7, 68, 71
 non-recognition 53
aging cat 112
agouti 13
Alia 3, 4
allergies 88
American Pet Products Manufacturer's Association 1
amusement 60
anesthetic 100
animal shelters 7, 9, 11, 19, 109, 125
 statistics 1
antibiotics 108

Ordered Index Structures

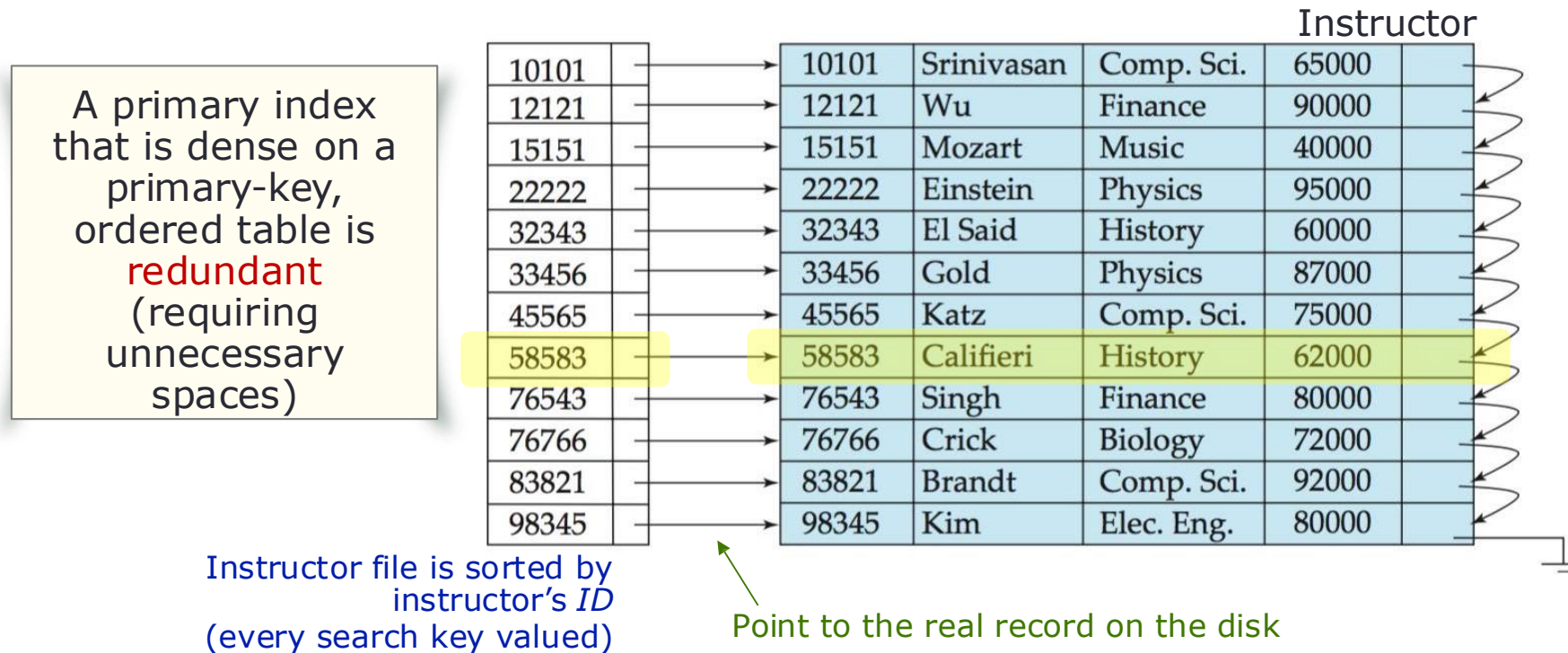
A file may have several indices, on different search keys

- **Primary index (or clustered index)**
 - Search key defines the sequential order of the file
 - Search key of a clustering index is often the primary key (but not necessarily so)
- **Secondary index (or unclustered index)**
 - Search key specifies an order different from the sequential order of the file
 - Use an extra-level of indirection to implement a secondary index, containing pointers to all the records

Dense Index

- A record is created for **every** search key value
- Need more space to store index records
- Example: **a search key is a primary key**

Find instructor
with ID "58583"

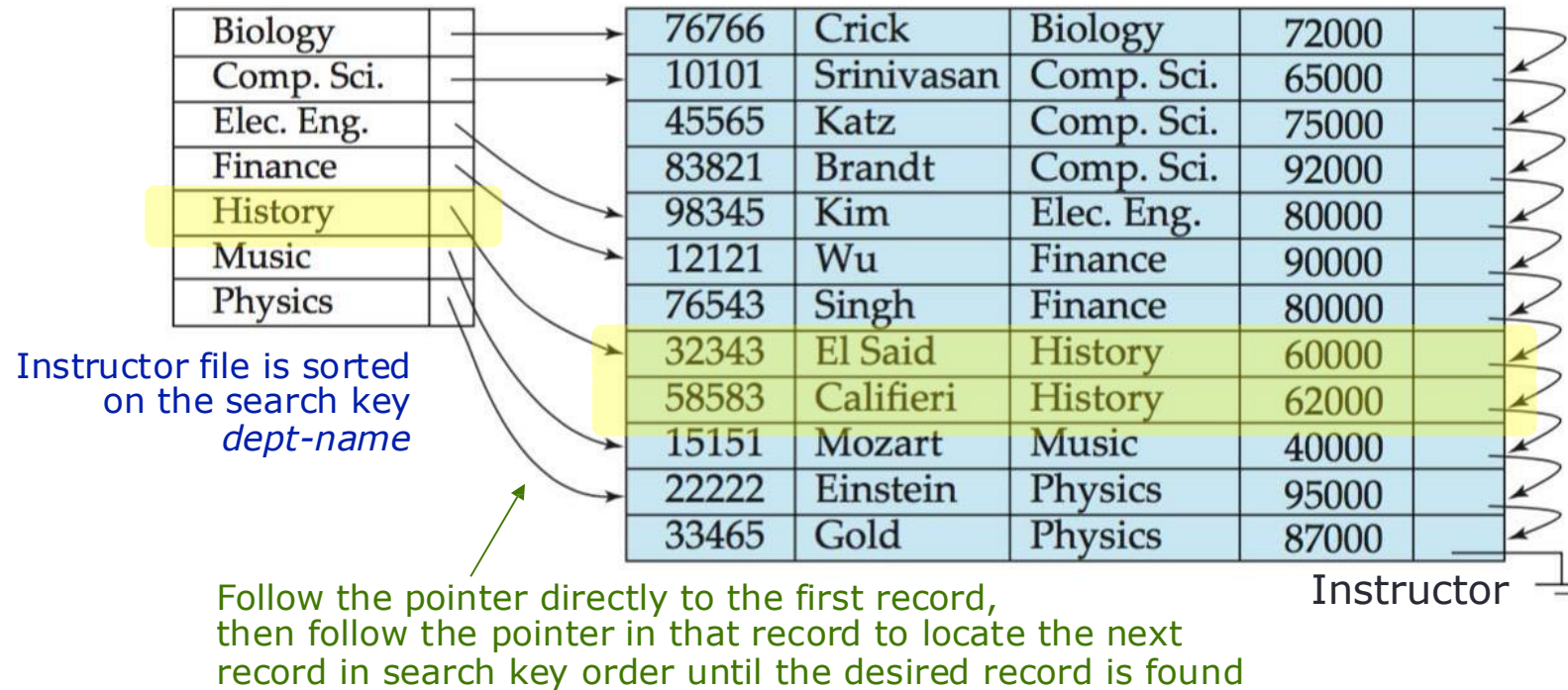


Dense Index

- Support range queries
- Example: a search key is not a primary key

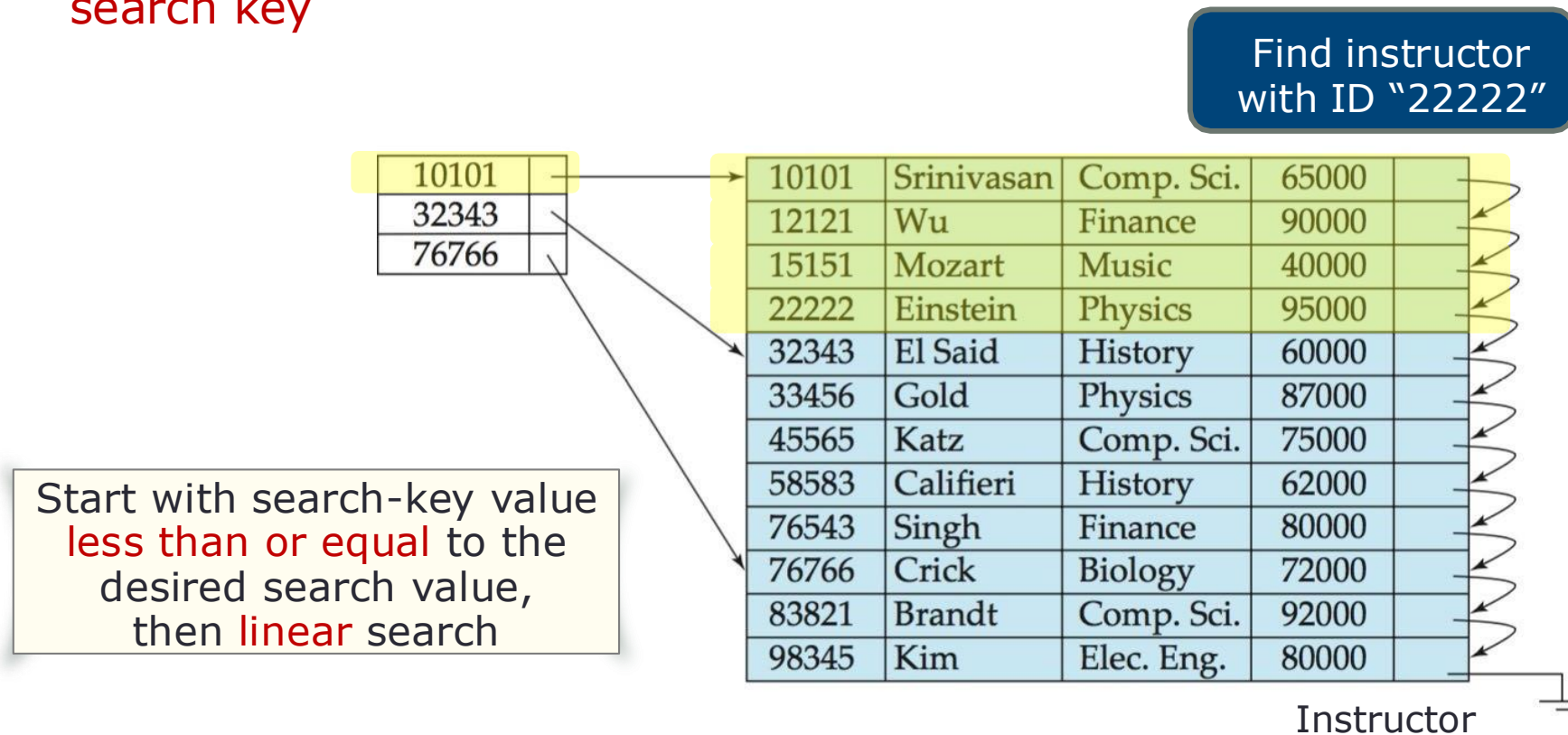
Find a history instructor with ID "58583"

Pointer points to the **first** data record with the search-value.
The rest of the records are sorted on the same search key



Sparse Index

- Used when dense indices are too large
- One key-pointer pair per data block
- Can be used only if the relation is stored in sorted order of the search key



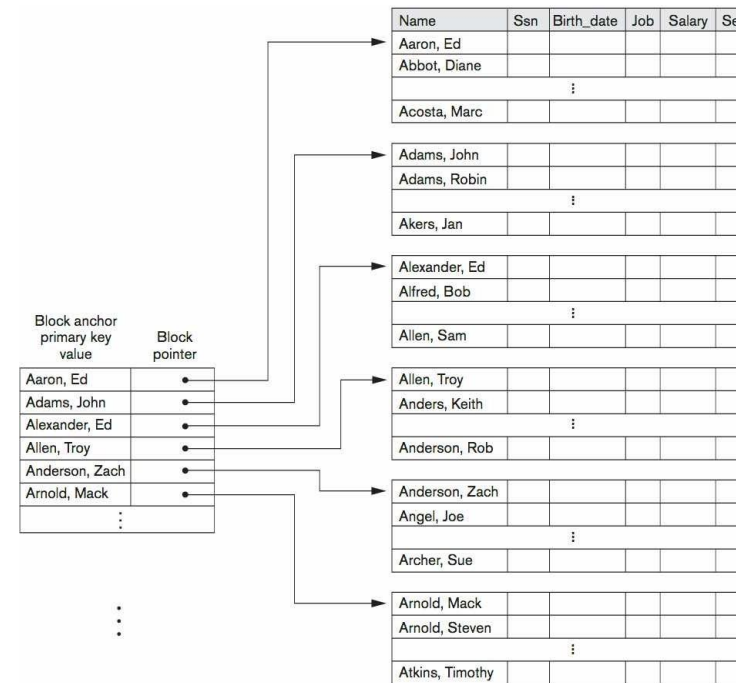
Sparse Index: Lookup

- Given a search key K ,
 - Search the sparse index for the greatest key $\leq K$, using binary search
 - Retrieve the pointed block to main memory to look for the record with search key K (linear search vs. binary search)
- The **index is usually kept in main memory**. Thus one disk I/O has to be performed during lookup
- **Efficient in space** but may **require more computation time** due to two binary searches
 - Search on the sparse index
 - Search on the retrieved data block

Dense Index vs. Sparse Index

- A primary index that is dense on an ordered table is **redundant**
- Thus, a primary index on an ordered table is always **sparse**
- **Dense** indices are **faster** in general
- **Sparse** indices require **less space** and impose **less maintenance** for insertions and deletions
- Try to have a sparse index with one entry per block

Try to keep index size small



How to Implement: Use B+ Trees

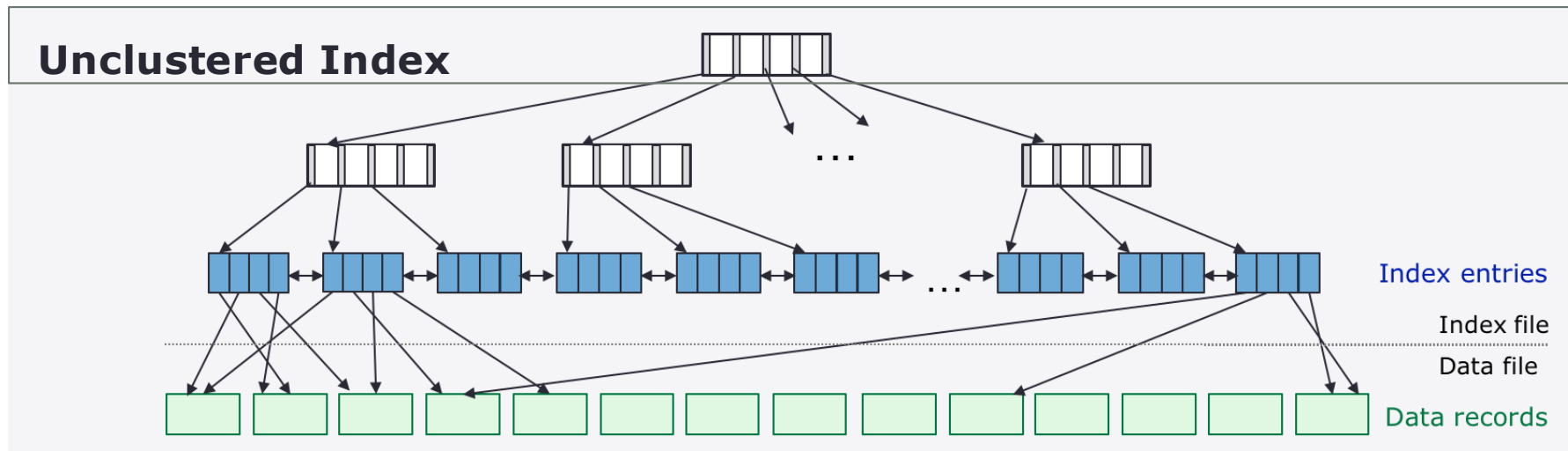
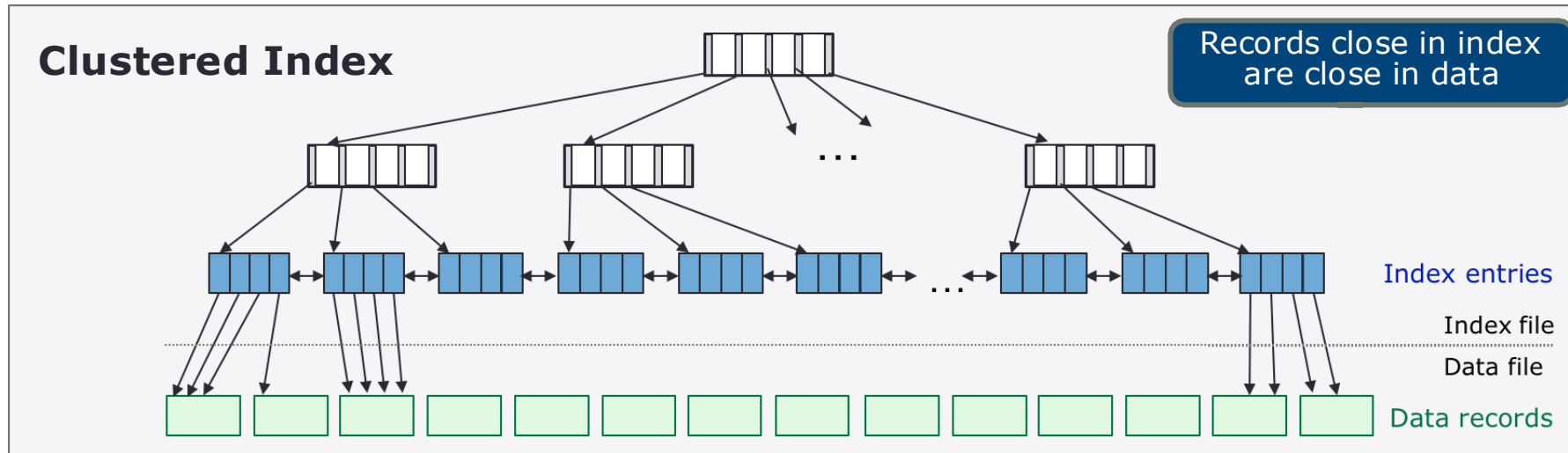
- As the database grows (and the index file grows), performance degrades. Reorganization is costly
- Solution: use B+ tree to maintain indices

B+ Tree

Balance sorted tree, allowing fast search and maintenance without overflow pages

- A tree-like file structure
 - Links nodes with pointers
 - Has exactly one **root**, bounded by **leaves**
 - Has unique path from root to each leaf; all paths are equal length
 - Store **keys** only at **leaves**, **references** in **other/internal nodes**
 - Guides key search via the reference values, from root to leaves
- **Balance** – same length on every path from root to leaves
- **Extensible** – number of pointers (n) at any given node

Clustered vs. Unclustered Index

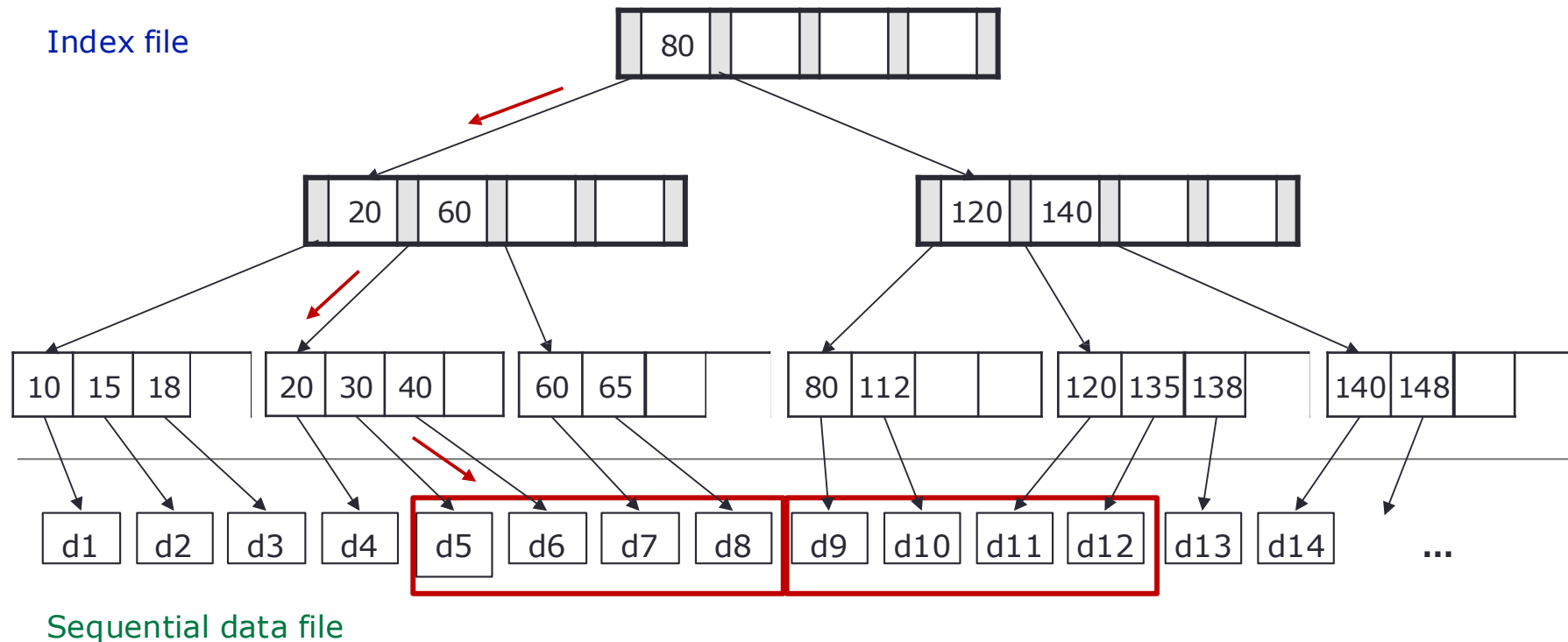


Cost of Disk I/O Operations

Assume a disk block holds 4 tuples of a relation. To find tuples associated with key values 40 – 85

Use clustered index, index and data are **sorted the same way**

Estimated cost = selectivity estimate x #blocks

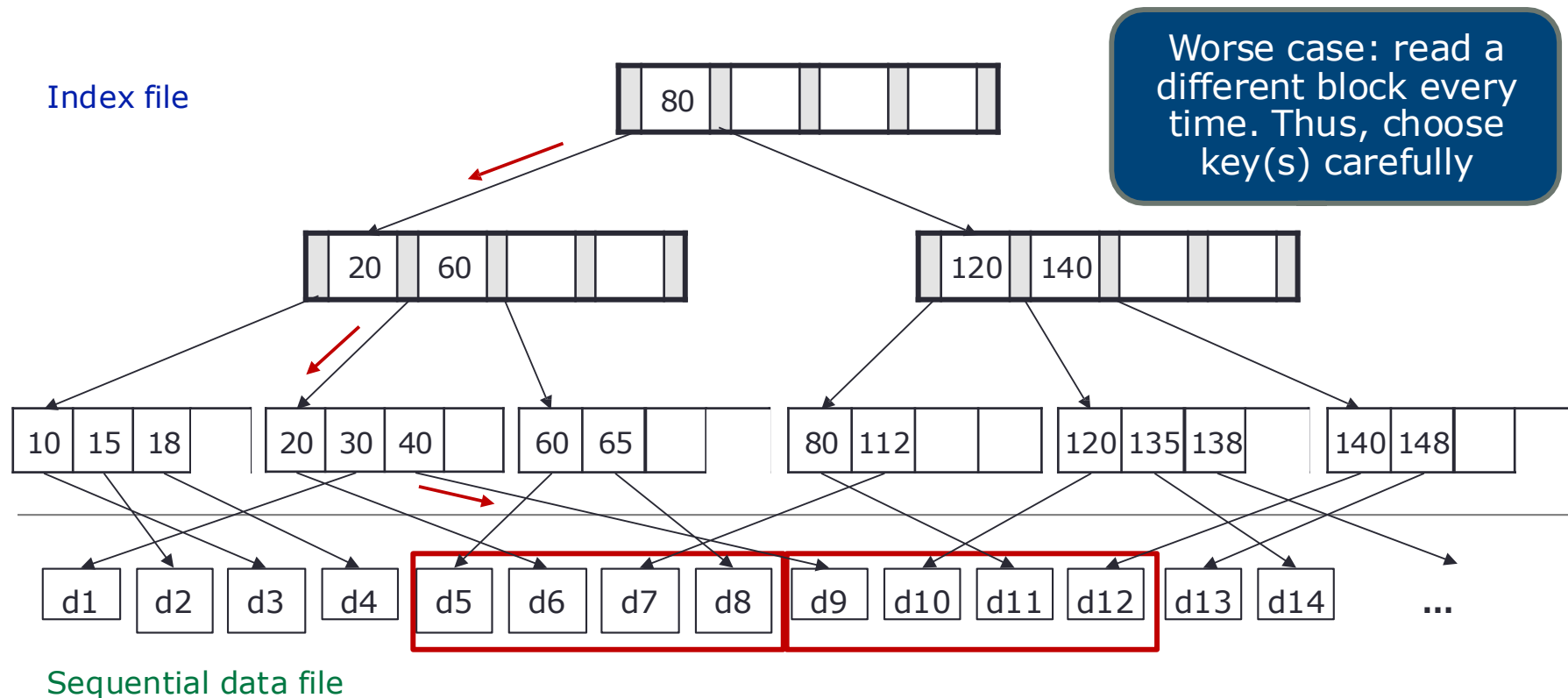


Cost of Disk I/O Operations

Assume a disk block holds 4 tuples of a relation. To find tuples associated with key values 40 – 85

Use unclustered index, index and data are sorted differently

Estimated cost = selectivity estimate x #tuples



Another Scenario

Supposed we know the following

#blocks = 100

#tuples = 10000

Using an unclustered index
in the wrong scenario can
lead to low performance

X = Selectivity estimate ($\sim$ proportion of tuples matching the selection) of a given query = 0.1

No index (full sequential scan)

Cost = #blocks
= 100

Unclustered index

Cost = X * #tuples
= 0.1 * 10000
= 1000

Full sequential scan is a better option when

- Selectivity is high (many tuples match the selection), or
- Ratio between #tuples:#blocks is high